Estrutura base

Referência:
https://docs.python.org/pt-br/3.15/tutorial/index.html

4. Mais ferramentas de controle de fluxo

Assim como a instrução while que acabou de ser apresentada, o Python usa mais algumas que encontraremos neste capítulo.

4.1. Instruções if

Provavelmente a mais conhecida instrução de controle de fluxo é o if. Por exemplo:

```
x = int(input("Insira um número inteiro: "))
```

```
Insira um número inteiro: 42
```

```
if x < 0:
    x = 0
    print('Negativo alterado para zero')
elif x == 0:
    print('Zero')
elif x == 1:
    print('Um')
else:
    print('Mais')
```

```
Mais
```

Pode haver zero ou mais partes elif, e a parte else é opcional. A palavra-chave 'elif' é uma abreviação para 'else if', e é útil para evitar indentação excessiva. Uma sequência if ... elif ... elif ... substitui as instruções switch ou case, encontrados em outras linguagens.
Se você está comparando o mesmo valor com várias constantes, ou verificando por tipos ou atributos específicos, você também pode achar a instrução match útil. Para mais detalhes veja Instruções match.

4.2. Instruções for

A instrução for em Python é um pouco diferente do que costuma ser em C ou Pascal. Ao invés de sempre iterar sobre uma progressão aritmética de números (como no Pascal), ou permitir ao usuário definir o passo de iteração e a condição de parada (como C), a instrução for do Python itera sobre os itens de qualquer sequência (seja uma lista ou uma string), na ordem que aparecem na sequência. Por exemplo:

```
# Mede algumas strings:
palavras = ['gato', 'janela', 'defenestrar']
for p in palavras:
    print(p, len(p))
```

```
gato 4
janela 6
defenestrar 11
```

Código que modifica uma coleção sobre a qual está iterando pode ser inseguro. No lugar disso, usualmente você deve iterar sobre uma cópia da coleção ou criar uma nova coleção:

```
# Cria uma amostra de coleção
users = {'Hans': 'active', 'Éléonore': 'inactive', '景太郎': 'active'}

# Estratégia: iterar por uma cópia
for user, status in users.copy().items():
    if status == 'inactive':
        del users[user]

# Estratégia: criar uma nova coleção
active_users = {}
for user, status in users.items():
    if status == 'active':
        active_users[user] = status
```

4.3. A função range()

Se você precisa iterar sobre sequências numéricas, a função embutida range() é a resposta. Ela gera progressões aritméticas:

```
for i in range(5):
    print(i)
```

```
0
1
2
3
4
```

O ponto de parada fornecido nunca é incluído na lista; range(10) gera uma lista com 10 valores, exatamente os índices válidos para uma sequência de comprimento 10. É possível iniciar o intervalo com outro número, ou alterar a razão da progressão (inclusive com passo negativo):

```
list(range(5, 10))
```

```
[5, 6, 7, 8, 9]
```

```
list(range(0, 10, 3))
```

```
[0, 3, 6, 9]
```

```
list(range(-10, -100, -30))
```

```
[-10, -40, -70]
```

Para iterar sobre os índices de uma sequência, combine range() e len() da seguinte forma:

```
a = ['Maria', 'tinha', 'um', 'carneirinho']
for i in range(len(a)):
    print(i, a[i])
```

```
0 Maria
1 tinha
2 um
3 carneirinho
```

Na maioria dos casos, porém, é mais conveniente usar a função enumerate(), veja Técnicas de iteração.
Uma coisa estranha acontece se você imprime um intervalo:

```
range(10)
```

```
range(0, 10)
```

Em muitos aspectos, o objeto retornado pela função range() se comporta como se fosse uma lista, mas na verdade não é. É um objeto que retorna os itens sucessivos da sequência desejada quando você itera sobre a mesma, mas na verdade ele não gera a lista, economizando espaço.
Dizemos que um objeto é iterável, isso é, candidato a ser alvo de uma função ou construção que espera alguma coisa capaz de retornar sucessivamente seus elementos um de cada vez até ser esgotado. Nós vimos que a instrução for é um exemplo de construção, enquanto que um exemplo de função que recebe um iterável é sum():

```
sum(range(4))  # 0 + 1 + 2 + 3
```

```
6
```

Mais tarde, veremos mais funções que retornam iteráveis e tomam iteráveis como argumentos. No capítulo Estruturas de dados, iremos discutir list() em mais detalhes.

4.4. Instruções break e continue

A instrução break sai imediatamente do laço de repetição mais interno, seja for ou while:

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(f"{n} igual a {x} * {n//x}")
            break
```

```
4 igual a 2 * 2
6 igual a 2 * 3
8 igual a 2 * 4
9 equals 3 * 3
```

A instrução continue continua com a próxima iteração do laço:

```
for num in range(2, 10):
    if num % 2 == 0:
        print(f"Encontrado um número par {num}")
        continue
    print(f"Encontrado um número ímpar {num}")
```

```
Encontrado um número par 2
Encontrado um número ímpar 3
```

```
Encontrado um número par 4
Encontrado um número ímpar 5
Encontrado um número par 6
Encontrado um número ímpar 7
Encontrado um número par 8
Encontrado um número ímpar 9
```

4.5. Cláusulas else em laços

Em um laço for ou while a instrução break pode ser pareada com uma cláusula else. Se o laço terminar sem executar o break, a cláusula else será executada.
Em um laço for, a cláusula else é executada após o laço finalizar sua iteração final, ou seja, se não ocorrer nenhuma interrupção.
Em um laço while, ele é executado após a condição do laço se tornar falsa.
Em qualquer tipo de laço, a cláusula else **não** é executada se o laço foi encerrado por um break. Claro, outras maneiras de encerrar o laço mais cedo, como um return ou uma exceção levantada, também pularão a execução da cláusula else.
Isso é exemplificado no seguinte laço for, que procura por números primos:

```
for n in range(2, 10):
    for x in range(2, n):
        if n % x == 0:
            print(n, 'igual a', x, '*', n//x)
            break
    else:
        # a iteração passou direto sem encontrar um fator
        print(n, 'é um número primo')
```

```
2 é um número primo
3 é um número primo
4 igual a 2 * 2
5 é um número primo
6 igual a 2 * 3
7 é um número primo
8 igual a 2 * 4
9 igual a 3 * 3
```

(Sim, este é o código correto. Observe atentamente: a cláusula else pertence ao laço for, **não** à instrução if.)
Uma maneira de pensar na cláusula else é imaginá-la pareada com o if dentro do laço. Conforme o laço é executado, ele executará uma sequência como if/if/if/else. O if está dentro do laço, encontrado várias vezes. Se a condição for verdadeira, um break acontecerá. Se a condição nunca for verdadeira, a cláusula else fora do laço será executada.
Quando usado em um laço, a cláusula else tem mais em comum com a cláusula else de uma instrução try do que com a de instruções if: a cláusula else de uma instrução try é executada quando não ocorre exceção, e a cláusula else de um laço é executada quando não ocorre um break. Para mais informações sobre instrução try e exceções, veja Tratamento de exceções.

4.8. Definindo funções

Podemos criar uma função que escreve a série de Fibonacci até um limite arbitrário:

```
def fib(n):    # escreve série de Fibonacci menor que n
    """Imprime uma série de Fibonacci menor que n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

# Agora chamamos a função que acabamos de definir:
fib(2000)
```

```
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```

A palavra reservada def inicia a *definição* de uma função. Ela deve ser seguida do nome da função e da lista de parâmetros formais entre parênteses. As instruções que formam o corpo da função começam na linha seguinte e devem ser indentados.

Opcionalmente, a primeira linha do corpo da função pode ser uma literal string, cujo propósito é documentar a função. Se presente, essa string chama-se *docstring*. (Há mais informação sobre docstrings na seção Strings de documentação.) Existem ferramentas que utilizam docstrings para produzir automaticamente documentação online ou para imprimir, ou ainda, permitir que o usuário navegue interativamente pelo código. É uma boa prática incluir sempre docstrings em suas funções, portanto, tente fazer disso um hábito.

A *execução* de uma função cria uma nova tabela de símbolos para as variáveis locais da função. Mais precisamente, todas as atribuições de variáveis numa função são armazenadas na tabela de símbolos local; referências a variáveis são buscadas primeiro na tabela de símbolos local, em seguida na tabela de símbolos locais de funções delimitadoras ou circundantes, depois na tabela de símbolos global e, finalmente, na tabela de nomes da própria linguagem. Embora possam ser referenciadas, variáveis globais e de funções externas não podem ter atribuições (a menos que seja utilizada a instrução global, para variáveis globais, ou nonlocal, para variáveis de funções externas).

Os parâmetros reais (argumentos) de uma chamada de função são introduzidos na tabela de símbolos local da função no momento da chamada; portanto, argumentos são passados *por valor* (onde o *valor* é sempre uma *referência* para objeto, não o valor do objeto). [1] Quando uma função chama outra função, ou chama a si mesma recursivamente, uma nova tabela de símbolos é criada para tal chamada.

Uma definição de função associa o nome da função com o objeto função na tabela de símbolos atual. O interpretador reconhece o objeto apontado pelo nome como uma função definida pelo usuário. Outros nomes também podem apontar para o mesmo objeto função e também pode ser usados pra acessar a função:

```
fib
```

```
<function fib at 10042ed0>
```

```
f = fib
f(100)
```

```
0 1 1 2 3 5 8 13 21 34 55 89
```

Conhecendo outras linguagens, pode-se questionar que `fib` não é uma função, mas um procedimento, pois ela não devolve um valor. Na verdade, mesmo funções que não usam a instrução return devolvem um valor, ainda que pouco interessante. Esse valor é chamado `None` (é um nome embutido). O interpretador interativo

evita escrever None quando ele é o único resultado de uma expressão. Mas se quiser vê-lo pode usar a função print():

```
fib(0)
print(fib(0))
None
```

É fácil escrever uma função que retorna uma lista de números da série de Fibonacci, ao invés de exibi-los:

```
def fib2(n):  # retorna série de  Fibonacci até n
    """Retorna uma lista contendo a série de Fibonacci até n."""
    resultado = []
    a, b = 0, 1
    while a < n:
        resultado.append(a)    # veja abaixo
        a, b = b, a+b
    return resultado

f100 = fib2(100)    # chama-o
f100                # escreve o resultado
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Este exemplo demonstra novos recursos de Python:

- A instrução return finaliza a execução e retorna um valor da função. return sem qualquer expressão como argumento retorna None. Atingir o final da função também retorna None.
- A instrução result.append(a) chama um *método* do objeto lista result. Um método é uma função que 'pertence' a um objeto, e é chamada obj.nomemetodo, onde obj é um objeto qualquer (pode ser uma expressão), e nomemetodo é o nome de um método que foi definido pelo tipo do objeto. Tipos diferentes definem métodos diferentes. Métodos de diferentes tipos podem ter o mesmo nome sem ambiguidade. (É possível definir seus próprios tipos de objetos e métodos, utilizando *classes*, veja em Classes) O método append(), mostrado no exemplo é definido para objetos do tipo lista; adiciona um novo elemento ao final da lista. Neste exemplo, ele equivale a result = result + [a], só que mais eficiente.

5.1. Mais sobre listas

O tipo de dado lista tem ainda mais métodos. Aqui estão apresentados todos os métodos de objetos do tipo lista:

list.**append**(*value*, */*)

Adiciona um item ao fim da lista. Similar a a[len(a):] = [x].

list.**extend**(*iterable*, */*)

Estende a lista, adicionando no fim todos os elementos do argumento iterável passado como parâmetro. Similar a a[len(a):] = iterable.

list.**insert**(*index*, *value*, */*)

Insere um item em uma dada posição. O primeiro argumento é o índice do elemento antes do qual será feita a inserção, assim a.insert(0, x) insere um elemento na frente da lista e a.insert(len(a), x) e equivale a a.append(x).

list.**remove**(*value*, */*)

Remove o primeiro item encontrado na lista cujo valor é igual a *value*. Se não existir valor igual, uma exceção ValueError é levantada.

list.**pop**(*index=-1*, */*)

Remove o item na posição fornecida na lista e retorna. Se nenhum índice for especificado, a.pop() remove e retorna o último item da lista. Levanta um IndexError se a lista estiver vazia ou o índice estiver fora do intervalo da lista.

list.**clear**()

Remove todos os itens de uma lista. Similar a del a[:].

list.**index**(*value*[, *start*[, *stop*]])

Retorna o índice de base zero da primeira ocorrência de *value* na lista. Levanta ValueError se não houver tal item.

Os argumentos opcionais *start* e *end* são interpretados como nas notações de fatiamento e são usados para limitar a busca para uma subsequência específica da lista. O índice retornado é calculado relativo ao começo da sequência inteira e não referente ao argumento *start*.

list.**count**(*value*, */*)

Devolve o número de vezes em que *value* aparece na lista.

list.**sort**(*, *key=None*, *reverse=False*)

Ordena os itens na lista (os argumentos podem ser usados para personalizar a ordenação, veja a função sorted() para maiores explicações).

list.**reverse**()

Inverte a ordem dos elementos na lista.

list.**copy**()

Devolve uma cópia rasa da lista. Similar a a[:].

Um exemplo que usa a maior parte dos métodos das listas:

```
frutas = ['laranja', 'maçã', 'pera', 'banana', 'kiwi', 'maçã', 'banana']
frutas.count('maçã')
2
frutas.count('tangerina')
0
frutas.index('banana')
3
frutas.index('banana', 4) # Encontra a próxima banana iniciando da posição 4
6
frutas.reverse()
frutas
['banana', 'maçã', 'kiwi', 'banana', 'pera', 'maçã', 'laranja']
frutas.append('uva')
frutas
['banana', 'maçã', 'kiwi', 'banana', 'pera', 'maçã', 'laranja', 'uva']
frutas.sort()
frutas
['maçã', 'maçã', 'banana', 'banana', 'uva', 'kiwi', 'laranja', 'pera']
frutas.pop()
'pera'
```

Você pode ter percebido que métodos como insert, remove ou sort, que apenas modificam a lista, não têm valor de retorno impresso – eles retornam o None padrão. [1] Isto é um princípio de design para todas as estruturas de dados mutáveis em Python.

Outro aspecto que você pode notar é que nem todos os dados podem ser classificados ou comparados. Por exemplo, `[None, 'hello', 10]` não é ordenável porque os inteiros não podem ser comparados a strings e `None` não pode ser comparado a outros tipos. Além disso, há alguns tipos que não têm uma relação de ordenação definida. Por exemplo, `3+4j < 5+7j` não é uma comparação válida.

5.7. Mais sobre condições

As condições de controle usadas nas instruções `while` e `if` podem conter quaisquer operadores, não apenas comparações.
Os operadores de comparação `in` e `not in` fazem testes de inclusão que determinam se um valor está (ou não está) em um contêiner. Os operadores `is` e `is not` comparam se dois objetos são realmente o mesmo objeto. Todos os operadores de comparação possuem a mesma prioridade, que é menor que a prioridade de todos os operadores numéricos.
Comparações podem ser encadeadas: Por exemplo `a < b == c` testa se `a` é menor que `b` e também se `b` é igual a `c`.
Comparações podem ser combinadas através de operadores booleanos `and` e `or`, e o resultado de uma comparação (ou de qualquer outra expressão), pode ter seu valor booleano negado através de `not`. Estes possuem menor prioridade que os demais operadores de comparação. Entre eles, `not` é o de maior prioridade e `or` o de menor. Dessa forma, a condição `A and not B or C` é equivalente a `(A and (not B)) or C`. Naturalmente, parênteses podem ser usados para expressar o agrupamento desejado.
Os operadores booleanos `and` e `or` são operadores *curto-circuito*: seus argumentos são avaliados da esquerda para a direita, e a avaliação encerra quando o resultado é determinado. Por exemplo, se `A` e `C` são expressões verdadeiras, mas `B` é falsa, então `A and B and C` não chega a avaliar a expressão `C`. Em geral, quando usado sobre valores genéricos e não como booleanos, o valor do resultado de um operador curto-circuito é o último valor avaliado na expressão.
É possível atribuir o resultado de uma comparação ou outra expressão booleana para uma variável. Por exemplo:

```
string1, string2, string3 = '', 'Trondheim', 'Hammer Dance'
non_null = string1 or string2 or string3
non_null
'Trondheim'
```

Observe que no Python, ao contrário de C, a atribuição dentro de expressões deve ser feita explicitamente com o operador morsa `:=`. Isso evita uma classe comum de problemas encontrados nos programas C: digitar `=` em uma expressão quando `==` era o planejado.

6. Módulos

Ao sair e entrar de novo no interpretador Python, as definições anteriores (funções e variáveis) são perdidas. Portanto, se quiser escrever um programa maior, será mais eficiente usar um editor de texto para preparar as entradas para o interpretador, e executá-lo usando o arquivo como entrada. Isso é conhecido como criar um *script*. Se o programa se torna ainda maior, é uma boa prática dividi-lo em arquivos menores, para facilitar a manutenção. Também é preferível usar um arquivo separado para uma função que você escreveria em vários programas diferentes, para não copiar a definição de função em cada um deles.
Para permitir isso, Python tem uma maneira de colocar as definições em um arquivo e então usá-las em um script ou em uma execução interativa do interpretador. Tal arquivo é chamado de *módulo*; definições de um

módulo podem ser *importadas* para outros módulos, ou para o módulo *principal* (a coleção de variáveis a que você tem acesso num script executado como um programa e no modo calculadora).
Um módulo é um arquivo contendo definições e instruções Python. O nome do arquivo é o nome do módulo acrescido do sufixo .py. Dentro de um módulo, o nome do módulo (como uma string) está disponível como o valor da variável global __name__. Por exemplo, use seu editor de texto favorito para criar um arquivo chamado fibo.py no diretório atual com o seguinte conteúdo:

```
# Módulo de números de Fibonacci

def fib(n):
    """Escreve a série de Fibonacci até n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):
    """Retorna a série de Fibonacci até n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

Agora, entre no interpretador Python e importe esse módulo com o seguinte comando:

```
import fibo
```

Isso não adiciona os nomes das funções definidas em fibo diretamente ao espaço de nomes atual (veja Escopos e espaços de nomes do Python para mais detalhes); isso adiciona somente o nome do módulo fibo.
Usando o nome do módulo você pode acessar as funções:

```
fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
fibo.__name__
'fibo'
```

Se você pretende usar uma função muitas vezes, você pode atribui-lá a um nome local:

```
fib = fibo.fib
fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

6.1. Mais sobre módulos

Um módulo pode conter tanto instruções executáveis quanto definições de funções e classes. Essas instruções servem para inicializar o módulo. Eles são executados somente na *primeira* vez que o módulo é encontrado em uma instrução de importação. [1] (Também rodam se o arquivo é executado como um script.)
Cada módulo tem seu próprio espaço de nomes privado, que é usado como espaço de nomes global para todas as funções definidas no módulo. Assim, o autor de um módulo pode usar variáveis globais no seu módulo sem se preocupar com conflitos acidentais com as variáveis globais do usuário. Por outro lado, se você precisar usar uma variável global de um módulo, poderá fazê-lo com a mesma notação usada para se referir às suas funções, `nomemodulo.nomeitem`.
Módulos podem importar outros módulos. É costume, porém não obrigatório, colocar todas as instruções import no início do módulo (ou script , se preferir). As definições do módulo importado, se colocados no nível de um módulo (fora de quaisquer funções ou classes), elas são adicionadas a espaço de nomes global da módulo.
Existe uma variante da instrução import que importa definições de um módulo diretamente para o espaço de nomes do módulo importador. Por exemplo:

```
from fibo import fib, fib2
fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isso não coloca o nome do módulo de onde foram feitas as importações no espaço de nomes local (assim, no exemplo, `fibo` não está definido).
Existe ainda uma variante que importa todos os nomes definidos em um módulo:

```
from fibo import *
fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isso importa todos as declarações de nomes, exceto aqueles que iniciam com um sublinhado (_). Na maioria dos casos, programadores Python não usam esta facilidade porque ela introduz um conjunto desconhecido de nomes no ambiente, podendo esconder outros nomes previamente definidos.
Note que, em geral, a prática do `import *` de um módulo ou pacote é desaprovada, uma vez que muitas vezes dificulta a leitura do código. Contudo, é aceitável para diminuir a digitação em sessões interativas.
Se o nome do módulo é seguido pela palavra-chave as, o nome a seguir é vinculado diretamente ao módulo importado.

```
import fibo as fib
fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isto efetivamente importa o módulo, da mesma maneira que `import fibo` fará, com a única diferença de estar disponível com o nome `fib`.
Também pode ser utilizado com a palavra-chave from, com efeitos similares:

```
from fibo import fib as fibonacci
fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Nota For efficiency reasons, each module is only imported once per interpreter session. Therefore, if you change your modules, you must restart the interpreter – or, if it's just one module you want to test interactively, use `importlib.reload()`, e.g. `import importlib; importlib.reload(modulename)`.

6.1.1. Executando módulos como scripts

Quando você rodar um módulo Python com
python fibo.py <argumentos>

o código no módulo será executado, da mesma forma que quando é importado, mas com a variável __name__ com valor "__main__". Isto significa que adicionando este código ao final do seu módulo:

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

você pode tornar o arquivo utilizável tanto como script quanto como um módulo importável, porque o código que analisa a linha de comando só roda se o módulo é executado como arquivo “principal”:

```
python fibo.py 50
0 1 1 2 3 5 8 13 21 34
```

Se o módulo é importado, o código não é executado:

```
import fibo
```

Isso é frequentemente usado para fornecer uma interface de usuário conveniente para um módulo, ou para realizar testes (rodando o módulo como um script executa um conjunto de testes).

10.8. Data e hora

O módulo datetime fornece classes para manipulação de datas e horas nas mais variadas formas. Apesar da disponibilidade de aritmética com data e hora, o foco da implementação é na extração eficiente dos membros para formatação e manipulação. O módulo também oferece objetos que levam os fusos horários em consideração.

```
# datas são facilmente construídas e formatadas
import datetime as dt
now = dt.date.today()
now
datetime.date(2003, 12, 2)
now.strftime("%m-%d-%y. %d %b %Y is a %A on the %d day of %B.")
'12-02-03. 02 Dec 2003 is a Tuesday on the 02 day of December.'

# datas têm suporte a matemática de calendário
birthday = date(1964, 7, 31)
age = now - birthday
age.days
14368
```